

Stratégie

Vision générale

Constat: dans Soak Overflow, la meilleure stratégie n'est pas un plan rigide et prédéfini. Elle émerge en fait d'elle-même lorsque chaque agent sélectionne la décision la plus pertinente selon :

- le contexte global de jeu
- son propre contexte individuel

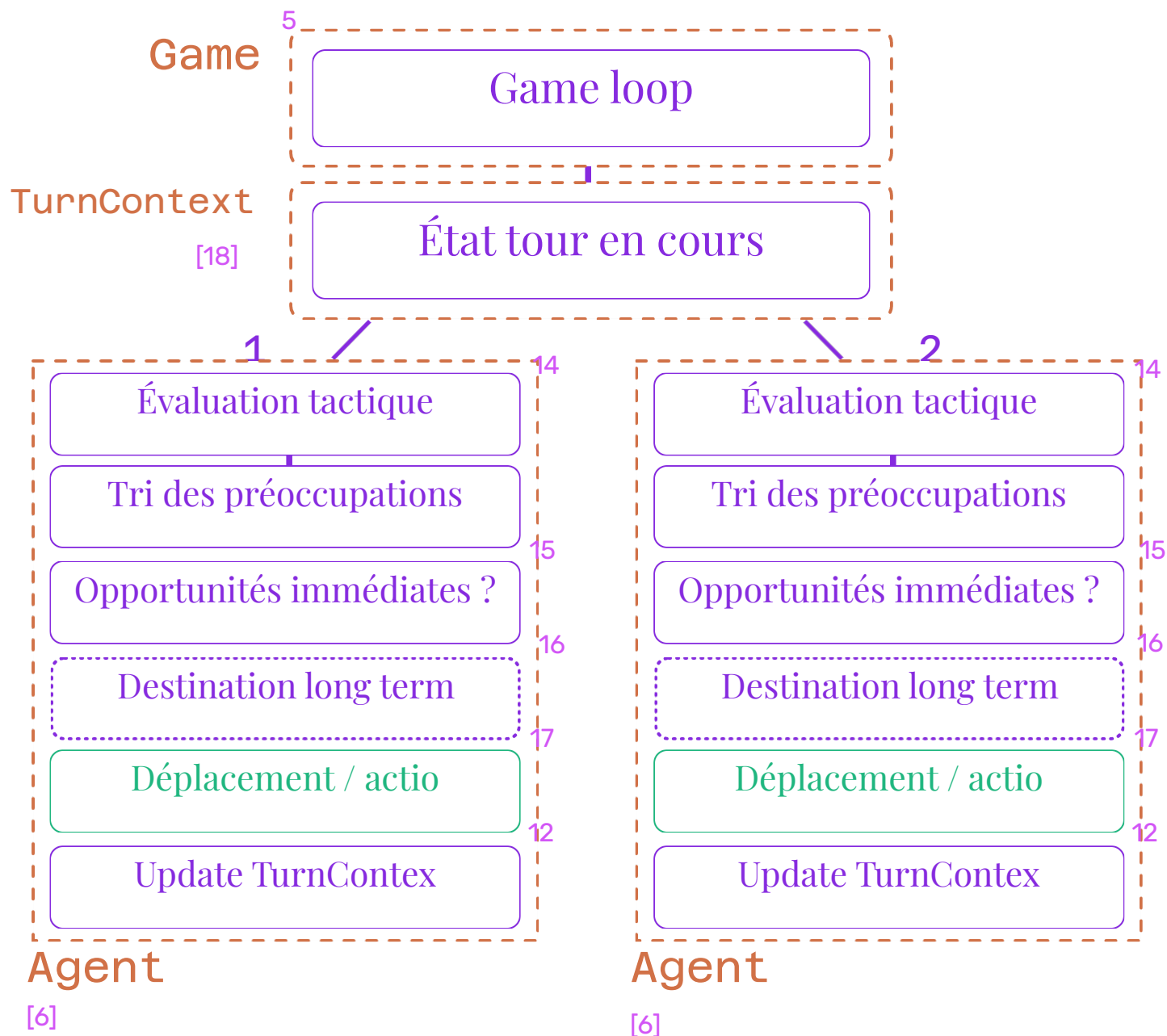
→ À chaque tour, c'est l'agent, et non le jeu, qui doit évaluer ses priorités.

Lire les principes de stratégie

4

□ Architecture

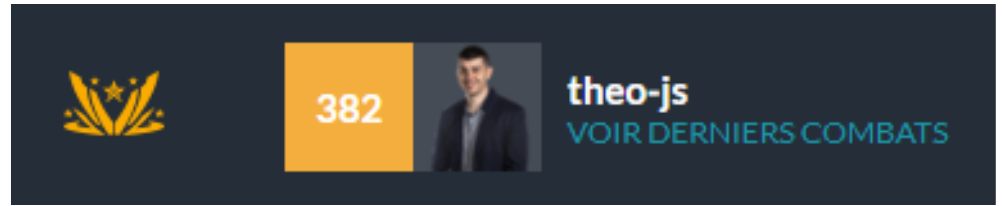
Vue d'ensem



Voir les détails d'implémentation

Conception et résultats

Classement



Ligue or atteinte

Démarche

Ce challenge a autant été un exercice d'algorithmique que de conception.

Mon objectif n'a pas été d'ajouter toujours plus de complexité, mais de construire un système cohérent, évolutif et explicable.

[Voir mon approche complète](#)

21

Principes de stratégi

13

Décision individuelle



12

Coordination prévisionnelle



Game

Rôle

Centralise l'état global de la partie (indépendant du tour courant), orchestre le déroulement des tours.

Responsabilités

- Lecture des données de jeu initiales (`constructor()`) et du tour courant (`readTurn()`).
- Instanciation/mise à jour du **TurnContext** [18] et de la **GameMap** [8].
- Définition de l'ordre de décision des agents et lancement des décisions (`playTurn()` → **Agent** [6].`decideActions()`).

Agent

Rôle

Encapsule le moteur de décision d'un agent.

Responsabilités

- Évaluation tactique locale :
 - possibilités offensives [19];
 - détection des menaces [20].
- Pipeline de décision [13].

Gestion des couvertur

Énoncé

Soak Overflow possède un système d'**obstacles** protégeant des tirs, dont chaque camp peut tirer parti à son avantage. Chaque obstacle peut servir de **couverture** aux **4** cases adjacentes.

Définitions

- **Couverture**: case adjacente à un obstacle;
- **Zone de protection**: zone résultant de la relation entre la couverture et l'obstacle, et d'où la puissance des tirs est atténuée.

Objectif

Connaître à tt moment la zone de protection d'une case.

Implémentation

La configuration du terrain est connue dès le début du jeu
☐ on peut **calculer la liste des couvertures** une seule fois, à l'initialisation de la **GameMap** [8].

Fonctionnement de [buildCoverMap\(\)](#) :

1. Pour chaque case du terrain, **repérer les obstacles**;
2. Pour chaque obstacle, **repérer ses cases adjacentes**;
3. Pour chaque adjacente, **calculer la zone de protection de l'autre côté** de l'obstacle;
4. Puisqu'une case peut être adjacente à **plusieurs** obstacles, on **fusionne les doublons** de la liste tout en **additionnant leur zones** de protection.

GameMap

Rôle

Centralise toute l'intelligence spatiale du jeu. Tous les calculs sur la carte sont regroupés dans cette classe.

Responsabilités

- Gérer la géométrie de la carte;
- Construire la liste de toutes les couvertures [7] possibles;
- Calculer les plus courts chemins [9];
- Évaluer le contrôle du territoire pour un placement d'agents donné.

Recherche de che

Objectif

Optimiser les déplacements.

Implémentation

Implémentation d'un [algorithme de parcours en largeur \(BFS\)](#) pour trouver le chemin le + court entre 2 positions du terrain, en tenant compte des obstacles et des positions déjà occupée par les agents.

AgentActionService

Rôle

Implémente la construction et la résolution des actions d'un agent.

Enregistrement des actions

Les différentes étapes d'un moteur de décision peuvent-enregistrer leurs intentions sans se préoccuper des conflits.

Résolution

Au moment de l'exécution, le service conserve :

- le dernier déplacement;
- la dernière action de combat;
- tous les messages

Intérêt

- L'usage est typé et idiomatique.
- Le protocole de sortie du jeu est géré à part dans une couche d'abstraction découplée du moteur de décision.

Détails d'implémentation

Principes généraux

- Séparation des responsabilités
- Pré-calcul favorisé & fail-fast → économie de ressources

Classes principales

Game



5

GameMap



8

TurnContext



18

Agent



6

AgentActionService



10

[Voir le code sur github](#)

Coordination prévisionnelle

Constat

Dans Soak Overflow, **chaque décision** influence directement l'issue du combat.

Sans réelle coordination, les agents peuvent prendre des décisions optimales d'un point de vue individuel, mais en réalité contreproductives car **conflictuelles**.

Ex:

- plusieurs agents gaspillent leurs ressources en bombardant le même ennemi mort en 1 coup.
- un agent tente de rejoindre la même case qu'un allié, ou une case touchée par une bombe alliée.

Solution : simulation de l'état au fil des décision

Tout au long du tour courant, une **simulation** de l'état des alliés et ennemis (positions, santé, bombes, etc.) est **mise à jour** ^[17] **après chaque décision**.

L'allié suivant peut alors prendre sa décision en tenant compte des effets de la décision précédente, ce qui évite les conflits.

N.B.: il y a une marge d'erreur inévitable, car la simulation est en partie spéculative (ex: dégâts de tirs peuvent être en réalité réduits par un recroquevillement ennemi difficile à prédire).

Optimisation : priorisation des décisions

Puisque les décisions antérieures peuvent impacter les suivantes, les **agents ayant le + fort potentiel tactique** doivent décider en premier (proches des ennemis, kill immédiat possible, etc.).

Décision individuelle

Objectif

Maximiser la valeur tactique d'une décision.

Contexte

- Lors d'un tour, les alliés prennent leur décision **l'un après l'autre**.
- Chaque allié contient propre son moteur de décision dont voici les **étapes internes** :

14

1. Évaluation tactique et ordre des priori



15

2. Recherche d'opportunité immédia



16

3. Validation de pertinence et naviga



17

4. Exécution de la décision



1. Évaluation tactique et ordre des priori

Principe

- **Tuer** un max d'ennemis; █
- **Infliger** un max de dégâts; █
- **Survivre**; █
- **Subir** le moins de dégâts; █
- Gagner / conserver le **territoire**; ████████

Toutes ces préoccupations sont essentielles à la victoire. Je suis parti du principe qu'**aucune ne devrait être négligée** pour mener à une décision. Elles doivent simplement être **hiérarchisées**.

Pour déterminer cet ordre de priorité, l'agent **examine** à la fois le contexte **global** de la partie, et sa situation **personnelle**.

À la fin de cette 1ère étape, nous avons établi la hiérarchie des préoccupations de notre agent durant ce tour.

Logique d'évaluation actuell

- | | |
|---|--------------|
| 1. Gravement blessé | █ ██████████ |
| 2. Infériorité numérique | █ ██████████ |
| 3. Ennemis hors de portée,
ou risque de défaite par perte territoire | █ █████ |
| 4. Ordre par défaut | █ █████ |

2. Recherche d'opportunité immédia

Principe

Rappel: un agent ne se déplace que de **max 1 case** par tour.
Si un combo déplacement/action intéressant est immédiatement actionnable dès ce tour, alors il doit l'être.

Simulation des décisions possibles et évaluati

Pour **chaque déplacement/action possible ici**, l'agent **mesure**:

- les **menaces** auxquelles il s'exposerait;
(dégâts potentiels totaux, probabilité d'être tué)
- la taille du **territoire** que l'équipe posséderait;
- les opportunités d'**attaque**.
(dégâts potentiels totaux, ennemis tuables)

Cette liste de combos évalués se veut exhaustive.

Ex: les lancers de bombe ne seront pas simulés pour un agent qui n'en possède pas.

Tri composé des évaluations

La liste des combos **passé à travers** un **tri dont l'ordre des critères correspond à celui de la hiérarchie des préoccupations** établie à l'**étape 1** ^[14].

À la fin de cette étape, nous obtenons donc un combo déplacement/action correspondant le plus aux priorités de l'agent pour ce tour (le premier de la liste triée).

3. Validation de pertinence et naviga

Principe

Un agent ne possède **pas toujours de bonne opportunité immédiate**. Lorsque c'est le cas, il vaut mieux rechercher une **destination** idéale sur la carte, et se déplacer vers elle.

Seuil de tolérance : un rapide contrôle qua

On veut **vérifier** que l'opportunité obtenue précédemment en est vraiment une, en imposant un **niveau de satisfaction minimal**, arbitraire, sous lequel l'opportunité sera rejetée.

Ici, le critère correspond uniquement à la **1^{ière} préoccupation** de l'agent pour ce tour.

Si l'opportunité passe la validation → passage direct **étape suivante** [17])

Opportunité rejetée » recherche de dest

Toute la carte de jeu est **scannée** à la recherche de la case correspondant le mieux à la **1^{ière} préoccupation** (ici aussi). À pertinence égale, la destination la + proche sera choisie.

L'agent cherche le + court chemin vers cette destination (algorithme de **BFS** [9]) et obtient le **premier pas** de ce chemin.

Le **combo final** est donc constitué :

- du **premier pas vers la destination** en tant que déplacement;
- de l'**action prévue par la meilleure opportunité** immédiate ayant également prévu de se déplacer vers cette case

4. Exécution de la décision

Principe

1. L'agent **exécute** ^[10] **la décision retenue** (mouvement et action).
2. Il **met à jour** ^[12] l'état du tour courant selon les effets attendus de sa décision, et passe le relais au prochain allié à décider.

TurnContext

Rôle

Centralise les données qui changent à chaque tour.

Responsabilités

- Stocke les données fiables reçues du jeu à chaque tour (agents)
- Expose des propriétés calculées (alliés/ennemis, scores)
- Expose un snapshot des agents muté au fil des décisions des alliés, utilisé pour éviter les conflits de décisions [12].

Évaluation des cibles (tirs et bombe)

Définitions

Une **cible** est :

- Pour une bombe, une position d'impact affectant un ou plusieurs ennemis;
- Pour un tir, la position d'un ennemi.

Principe

Chaque victime de chaque cible atteignable est évaluée selon :

- les dégâts effectifs;
- l'attaque résultera-t-elle en un kill ?
- l'overkill (information exploitable pour optimiser l'économie des ressources de jeu)

Estimation des dégâts

Pour les tirs, le calcul est fonction de :

- la distance;
- la portée optimale et la puissance du tireur;
- le niveau de protection offert par les **couvertures** [7].

Les bombes font des dégâts fixes.

Approche fail-fast

Si l'état personnel de l'agent ne permet pas une attaque (cooldown pour les tirs et nbre en stock pour les bombes), aucun calcul n'est lancé.

Évaluation des menace_

Principe

Pour chaque ennemi, on mesure :

- les dégâts maximaux qu'il pourrait nous infliger;
- les dégâts résulteraient-t-il en un kill ?

Estimation des dégâts_

- La méthode de calcul des dégâts effectifs est la même que pour l'évaluation des cibles [19].
- On tient compte de l'état de l'ennemi (cooldown actif ? / bombes en stock ?)

Approche

Mes pratique



22

Itérations



25

Ce que j'ai appri



24

Améliorations possibles



26

Pratiques illustrées dans le dé

- **Typage** fort;
- **Simplification** des solutions;
- **Refactoring** régulier;
- Souci de **lisibilité** : nommage intentionnel;
- **Prévisibilité** : règles explicites plutôt que basées sur des scores/coefficients arbitraires;
- **Séparation** des responsabilités;
- **Automatisation** : challenge relancé automatiquement via **script devtools** [23].
- **Pragmatisme** : les résultats mesurés priment sur l'élégance.

Évaluation des résultats

Problème

La **variabilité des résultats** entre deux essais (jusqu'à 200 places d'écart au classement) rend impossible une **comparaison fiable** de l'efficacité de deux versions du programme.

Palliatif

J'ai écrit un court script de **relance automatique** du challenge, pour **augmenter la taille de l'échantillon**.

La moyenne obtenue sur un + grand nombre d'observations donne une estimation + fiable de l'efficacité réelle d'une solution.

Ce que j'ai appris

- Une solution ne doit pas être optimale dans l'absolu, c'est le **système** avec lequel elle interagit (ici les autres joueurs de la ligue) qui révèle son efficacité.
- On ne résout pas des problèmes complexes uniquement par **déduction**. L'observation et l'expérimentation la complètent.

Itérations

Pour progresser dans le classement, j'ai **mesuré**^[23] systématiquement l'impact de mes changements tout au long de la conception.

Comment je suis sorti de la stagnati

Vers la moitié de la ligue argent, mon programme a montré ses **limites**. La solution, qui me plaisait car je la trouvais **élégante** et **évolutive**, se basait sur des stratégies collectives paramétrant un moteur de décision individuelle léger.

Je me suis convaincu qu'en peaufinant la granularité dans le choix des stratégies collectives, le programme finirait par couvrir suffisamment de cas de figures pour atteindre l'or.

Mais face à l'absence de progression, j'ai tout remis en question et réalisé que le problème était + **structurel** : mon moteur de décision n'était simplement pas assez **intelligent**.

J'ai **supprimé** beaucoup de logique, compris où ma version péchait, mis en place des systèmes qui lui manquaient, et conçu un algorithme de prise de décision bien + **exhaustif** et **robuste**, ce qui m'a fait gagner des centaines de places au classement.

Améliorations possibles

Performance : le point que j'améliorerais en

Certaines opérations ont actuellement une haute **complexité** algorithmique, la + importante étant la recherche sur la carte de la position donnant le + grand contrôle du territoire.

Une solution de **précalcul** a été tentée, mais abandonnée car le résultat n'a pas pu être réutilisé efficacement.